A CLIPS Rule-Based Agent for Wumpus World

Project report — Artificial Intelligence and Laboratory, Planning and Rule Systems module, a.y. 2025–2026

Daniel Biasiotto

November 2026

Contents

Abstract 1
1. Introduction 1
2. Simulator and protocol 2
3. Agent architecture 3
4. Hazard inference 4
    4.1 Certainty factors 4
5. Planning and energy 4
6. The map1 dead end and the bounded risk probe 6
7. Pit guesses 6
8. Evaluation 6
9. Limitations and future work 8
10. Conclusion 8
Appendix: running the project 8

Abstract

This report describes `agent.clp`, a rule-based agent for the Wumpus World domain, connected through `clipspy` to a Python discrete-event simulator. The agent uses six CLIPS modules, classical hazard inference, explicit certainty factors combined with the MYCIN formula, distance-field planning over the visited graph, an energy-aware return policy, and a bounded last-resort risk probe for dead ends. On the committed 11-map suite it survives 10 maps, collects the gold on 9, returns to the exit on 10, and scores 975 points; the provided random baseline survives none, collects no gold, returns on none, and scores -1100. The agent also demonstrates pit hypotheses and `pit-*` actions.

1. Introduction

Wumpus World is a classic testbed for reasoning under uncertainty. The agent stands in a dark cave, perceives only local signals, and must decide where to move without seeing the whole map. The project has four goals:

1. survive the cave;
2. collect the gold;

3. return to the exit `(0, 0)` with energy to spare;
4. estimate where pits are and, when profitable, use the `pit-*` actions.

The work is split into three parts: a minimal alignment of the simulator and its Python/CLIPS bridge, the rule-based agent itself, and a reproducible evaluation against the random baseline. The code and documentation are in English. The agent is implemented only with standard CLIPS and `clipspy`; native certainty factors are not used because they are unavailable in CLIPS 6.41, the engine shipped with `clipspy 1.0.6`.

2. Simulator and protocol

The simulator is a discrete-event grid. Coordinates start at `(0, 0)` and grow down and right. Walls are cells, not edges, and may hide a pit. The Wumpus is unique and cannot share a cell with a pit, a wall, or the gold. The gold is unique and cannot share a cell with a pit or a wall. Percepts are four-neighbour and local: `breeze` means an adjacent pit, `stench` means an adjacent Wumpus, and `glitter` means the gold is in the current cell.

Action	Effect	Energy	Reward
`up/down/left/right`	move if the target is in bounds and not a wall; otherwise stay	−1; −2 on a bump	step cost
`grab`	collect the gold if present	−1	+100
`quit`	end the episode	−1	0
`pit-north/south/west/east`	test the adjacent cell, no movement	−1	+25 / −25

The bridge has a strict one-action protocol. Before the first `run()` the initial percept is asserted. After every step the old percept is retracted, the new percept is asserted, the counter is advanced, and the previous action is cleared. Exactly one `(action (move ?))` fact must be produced per cycle; zero or multiple actions raise a protocol error instead of silently choosing a direction. The harness also rejects a `quit` outside `(0, 0)`, even though the simulator itself accepts it anywhere.

`StepResult.reward` was aligned to return the accumulated score, while the scoring model (`+100` gold, `±25` guesses, `-100` death) is unchanged. A grab turn that exhausts energy is terminal and charges the death penalty once. The local `map1.json` uses `end_on_grab: false`, so the agent must return to the exit; the harness also supports `end_on_grab: true`, where the episode ends on the grab.

The simulator fixes were kept minimal and behavior-preserving. The GUI was moved out of `wumpus.py` so the simulator and the headless harness work without Tk. `WumpusWorld.step` was split into small helpers so every Python function stays within cyclomatic complexity grade B (≤ 10).

3. Agent architecture

The agent uses six modules, one per phase. `MAIN` owns the schema and the cycle clock; the other modules import `MAIN ?ALL`.

```
MAIN
  templates, counter, stale-percept cleanup, single dispatch
    |
    +--> PERCEPT   percept -> pos + visited
    +--> MAP       visited/blocked -> no-pit, no-wumpus, safe, nbr, frontier
    +--> HYPO      breeze/stench closure, candidates, pit/Wumpus, CF
    +--> DECIDE    home/dist/nav fields, energy budget, goal, next hop
    +--> ACT       exactly one action: grab, quit, guess, move
```

`MAIN::dispatch-cycle` fires only when no action exists and focuses `PERCEPT MAP HYPO DECIDE ACT`. The focus stack is one-way, so each phase only sees the facts produced by the phases before it. This is why `HYPO` asserts frontiers for newly inferred safe cells: without that rule, `DECIDE` would not see them until the next MAP sweep.

The central facts are:

Fact	Meaning
`percept`, `counter`, `action`	bridge input/output and clock
`pos`, `visited`	current and visited cells
`blocked`	wall or out-of-map cell discovered by a bump
`no-pit`, `no-wumpus`	classical exclusions
`pit`, `wumpus`	confirmed hazards
`safe`, `frontier`	hazard-free and safe-unvisited cells
`breeze-at`, `no-breeze-at`, `stench-at`, `no-stench-at`	percept evidence
`candidate`	a cell that can explain one breeze
`wumpus-possible`	a cell compatible with every stench
`pit-hypothesis`	explicit pit belief with a CF and a source cell
`home`, `dist`, `nav`	return, current-position, and goal distances
`goal`, `next`	current target and ranked next hop
`guessed`	pit cells already tested
`has-gold`, `probes-used`, `probe-attempted`	persistent mission state

`MAIN` uses `(not (action (move ?)))` on every dispatch and `ACT` uses the same guard on every action rule. No `halt` is used: `halt` leaves the agenda intact and can resume stale activations on the next run.

4. Hazard inference

The inference is classical and monotone. A visited cell is not a pit, not the Wumpus, and safe. A cell without a breeze rules out pits in every orthogonal neighbour, including walls and cells outside the map. A cell without a stench rules out the Wumpus in every neighbour. Negative coordinates are outside the map, so they are `blocked`, `no-pit`, and `no-wumpus`; they can never become safe because they cannot be entered.

A wall discovered by a bump is `blocked` and `no-wumpus`, but it is not automatically `no-pit`: a pit may share a wall cell. Only a no-breeze observation, or the geometric fact that the coordinate is negative, rules out a pit there.

For each breeze cell, every adjacent cell that is not `no-pit` and not the Wumpus is a candidate. Confirmed pits remain candidates: removing them would let a second cell look like the unique explanation and produce a false confirmation. A breeze with exactly one candidate confirms a pit.

The Wumpus is unique, so a cell is possible only if it is adjacent to **every** stench cell. The rule expresses "no stench cell lacks this neighbour" with a negated conjunction:

```
(not (and (stench-at (x ?ox) (y ?oy))
          (not (nbr (x ?ox) (y ?oy) (nx ?px) (ny ?py)))))
```

This avoids the derive/prune oscillation that a separate intersection rule would create. A single remaining `wumpus-possible` cell confirms the Wumpus. Once the Wumpus is confirmed, every other known cell is `no-wumpus`, which completes the uniqueness closure.

4.1 Certainty factors

CLIPS 6.41 has no native certainty-factor machinery: `(declare (CF ...))`, `(get-cf ...)`, and `(assert ... cf ...)` are syntax errors. The agent therefore uses explicit CF slots, the approach used by the course's `wine.clp` example.

Each candidate receives one `pit-hypothesis` fact per breeze source with `cf = 0.4`. The `source-x`/`source-y` slots are essential: CLIPS deduplicates identical facts, so three 0.4 hypotheses for the same cell would collapse into one before combination. `evidence-seen` prevents the same breeze from contributing twice on later cycles.

Hypotheses for the same target are combined with the MYCIN positive formula:

```
cf = cf1 + cf2 * (1 - cf1)
```

The operator is associative and commutative, so the order of pairwise combinations does not matter. One breeze gives `0.4`, two give `0.64`, three give `0.784`, and a confirmed pit is raised to `1.0`. Classical facts always win: a `no-pit` or Wumpus target drops the hypothesis, and CF never creates `safe` or `pit`.

5. Planning and energy

Planning uses three distance fields over the known graph:

- `home`: shortest distance to `(0, 0)` over **visited** cells only. It is exact for the path the agent actually walked and cannot be invalidated by a later bump. Frontiers are leaves, reached through a visited neighbour.
- `dist`: shortest distance from the current position over visited cells, with frontiers as leaves.
- `nav`: distance to the chosen goal. The goal is seeded at distance 0; a frontier is reached from its visited neighbours; expansion and relaxation run only over visited cells.

Distances use `expand`/`relax` rules. An `expand` rule asserts a distance for an unreached node; a `relax` rule lowers an existing distance only when the new value is smaller. Unit edge costs make this converge to the shortest path independently of activation order.

A frontier `f` is affordable only if

```
dist(f) + home(f) + 1 + reserve <= energy
```

where `+1` is the final quit and `reserve` is 5 energy. The goal priority is:

Priority	Goal	Condition
1	`grab`	glitter is true and the grab plus the return still leaves the reserve
2	`probe`	no safe frontier, no gold yet, a bounded risk candidate is affordable
3	`quit`	at `(0, 0)`, with gold, no affordable frontier, or low energy
4	`exit`	not at `(0, 0)`, with gold, low energy, or no affordable frontier
5	`frontier`	an affordable safe frontier exists

A grab requires `energy >= home + reserve + 2`: one unit for the grab, one per home step, one for quit, and the reserve. If the reserve would be broken, the agent returns without grabbing. `quit` is selected only at `(0, 0)`.

The next hop is a visited neighbour whose `nav` is exactly one less than the current cell, plus the goal frontier itself when adjacent. Ties use the fixed direction order `up=1`, `right=2`, `down=3`, `left=4` (north, east, south, west), so the route is deterministic.

A bump is detected without a bump percept: `ACT` records the intended move and its origin, and `MAP` compares it with the next position. If the position did not change, the target is `blocked`; if it changed, the move succeeded. One bump per unknown wall or border is accepted; a repeated bump is a planning error. The frontier is retracted after a bump and the next cycle replans.

6. The map1 dead end and the bounded risk probe

Strict safety leaves the gold on `map1.json` unreachable. The safe closure from the start is:

```
(0,0) (0,1) (0,2) (0,3) (0,4)
            (1,3) (1,4) (2,4)
```

The only connection to the rest of the cave is `(3,4)`. The visited cell `(2,4)` reports a breeze; its candidates are `(2,3)` (a confirmed pit), `(2,5)` (a real pit), and `(3,4)` (actually safe). Since extra pits are allowed, no sound rule can prove `(3,4)` safe. A strict agent must quit without the gold.

The project therefore includes a bounded, last-resort risk probe. It is deliberately narrow:

- it runs only when there is no safe frontier at all, the gold is not held, the agent is not standing on the gold, and the energy still covers the return;
- candidates are unvisited, not blocked, not confirmed Wumpus-possible, and have `cf <= 0.75`;
- at most two probes are allowed, and a second probe is unlocked only when the first one reached its target; a wall bump ends risk;
- candidates are ranked by the number of visited neighbours (known escape routes), then by CF, then by geometry.

The support-first order matters. On the second cut of map1, the safe `(4,3)` has two visited approaches and CF `0.64`, while the real pit `(5,4)` has one approach and CF `0.4`. Ranking by CF alone would choose the pit. The CF is not calibrated here because overlapping breezes inflate safe cells; support is a better proxy for a crossable cut.

The policy remains a gamble on a genuine pit. On `map1` the two probes `(3,4)` and `(4,3)` are safe, so the agent collects the gold and returns. On `map10` the probe enters a real pit and the agent dies; that outcome is kept in the evaluation instead of being hidden. The budget, threshold, and reserve are configurable.

7. Pit guesses

A `pit-*` action costs one energy and pays `+25` if the adjacent cell is a pit, `-25` otherwise. The expected value is:

```
EV = 25p - 25(1-p) - 1 = 50p - 26
```

which is positive for `p > 0.52`. The agent uses a conservative threshold of `cf >= 0.75`. It guesses only an adjacent cell, never a blocked, safe, Wumpus-possible, or already guessed cell, and only when the energy still covers the return plus one unit. It does not guess while holding the gold, while returning to the exit, or while a risk probe is in progress: those goals have higher priority. A `guessed` marker persists for the episode, so the same cell is never tested twice. The four direction mappings are `up -> pit-north`, `right -> pit-east`, `down -> pit-south`, and `left -> pit-west`.

8. Evaluation

The benchmark runs `run_episode` on `map1.json` plus ten committed maps in `maps/`. The suite varies size, walls, pit density, Wumpus placement, gold distance, and energy. Each episode records

survival, gold, return, reward, energy, steps, visited cells, guesses, and guess accuracy. The random baseline is reseeded after reset, so its episodes are deterministic under the chosen seed.

Command:

```
python tools/benchmark.py --json benchmark.json
```

Aggregate result (seed 7):

Agent	Maps	Survival	Gold	Return	Avg energy	Avg steps	Guesses	Accuracy	Reward
expert	11	90.9%	81.8%	90.9%	29.8	33.5	7/7	100%	975
random	11	0.0%	0.0%	0.0%	−0.1	43.3	0/0	—	−1100

Per-map expert outcomes:

Map	Size	Termination	Gold	Return	Reward	Energy
map1	8x6	quit	yes	yes	125	111
map2	4x4	quit	yes	yes	100	16
map3	5x5	quit	yes	yes	100	12
map4	6x5	quit	yes	yes	200	14
map5	6x5	quit	yes	yes	100	30
map6	8x4	quit	yes	yes	100	32
map7	5x5	quit	no	yes	0	5
map8	6x4	quit	yes	yes	100	26
map9	8x6	quit	yes	yes	125	47
map10	6x5	death	no	no	−100	0
map11	7x5	quit	yes	yes	125	35

`map7` is a survival-first outcome: the gold is unaffordable under the 14-step budget, so the agent returns and quits. `map10` is the documented risk trade-off: the bounded probe enters a real pit even though an alternative safe crossing exists. No protocol errors occurred.

The map1 episode is a useful concrete trace:

```
Termination: quit
Steps: 76
Reward: 125.0
Energy: 111
Gold collected: True
```

```
Final position: (0, 0)
Visited cells: 31
Guess pit-north -> (2, 3): correct
```

9. Limitations and future work

- The risk probe can die. It is a deliberate last-resort policy, not a proof of safety. A stricter setting (`max-probes 0`) gives survival-only behavior and no map1 gold.
- Certainty factors are heuristic. The evidence weight is fixed at 0.4 and overlapping breezes can inflate a safe cell. A calibrated model or a proper Bayesian treatment would be a natural extension.
- Pit guesses are adjacent-only. A confirmed pit that is never adjacent is not tested, even though the action is profitable.
- The `home` field is conservative: it only uses the visited graph, so the agent may prefer a longer known path over a shorter unproven one. This is intentional for safety.
- Upper map bounds are unknown. The agent discovers them by bumping, which costs energy; future work could learn the bounds earlier from repeated bumps.
- The agent does not learn from episode to episode. Each episode starts from a fresh CLIPS environment.

10. Conclusion

`agent.clp` is a complete, explainable CLIPS agent for the Wumpus World protocol. It combines classical hazard inference, explicit MYCIN certainty factors, distance-field planning, energy budgeting, a bounded risk policy, and pit guesses. The simulator and harness were aligned so the agent can be tested headlessly and deterministically. On the committed suite the expert survives 10 of 11 maps, collects the gold on 9, returns on 10, and substantially outperforms the random baseline. The two non-successes are explicit policy trade-offs rather than protocol failures.

Appendix: running the project

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements-dev.txt

# GUI
python wumpus.py -m map1.json -a agent.clp

# one episode
python tools/evaluate.py -m map1.json -a agent.clp

# full benchmark
python tools/benchmark.py

# tests and quality gates
pytest -q tests/
flake8 . --count --select=E9,F63,F7,F82 \
```

```
  --show-source --statistics --exclude=.venv,__pycache__,.git
radon cc -s -a -nb wumpus.py gui.py tools/ tests/
```

The test suite covers the simulator semantics, the Python/CLIPS bridge, the agent contract, perception and map closure, hazard inference and CF, planning and risk probes, pit guesses, the benchmark, and the GUI protocol errors. The CI workflow runs the same checks on Python 3.11.